honey-1

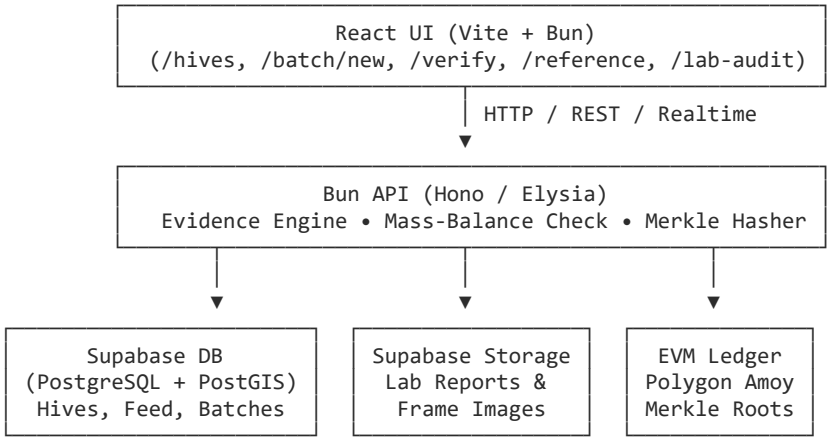
https://gemini.google.com/app/6f30d678c45a1652

User prompt: i have two doc, demo and real project. i have to make real project but i need to understand and take inspiration from somewhere so i am using demo project as inspiration (made by other team) . help me to plan the real project. (i will be using bun and react for development and supabase as db)

Response: Project Architecture & Comparison

The demo project (**Bee Bridge**) established a strong benchmark for consumer presentation, basic smart-contract batch anchoring, and an e-commerce marketplace. However, the core vulnerability in real-world honey traceability is **post-harvest substitution and syrup feeding**, where bad actors inject synthetic syrups or claim 100 kg of honey from a 20 kg harvest.

Your real project (**HoneyChain**) fixes this by introducing an **Evidence Engine**: cross-checking hive telemetry (weight deltas vs. feed logs) to enforce mass balance, maintaining sealed reference samples, and detecting tamper attempts directly at the consumer tap.



Layer	Demo Implementation (Bee Bridge)	Planned HoneyChain Implementation (Your Project)
Runtime & API	Node.js + Express	**Bun + Hono / Elysia** (Ultra-fast startup, native TypeScript, low latency)
Frontend UI	React + Tailwind (Single verification layout)	**React + Tailwind** (Hive weight monitors, Batch creator, 4-step Verification, Tamper alert)
Database & Auth	Supabase (PostgreSQL, standard tables)	**Supabase (PostgreSQL + RLS + Realtime)** with Mass-Balance & Split-Sample models
Traceability Logic	Static metadata hashed to smart contract	**Evidence Engine**: (Hive Δ mass $-$ Feed mass $\geq$ Harvested mass) + Merkle Tree on-chain root
Verification Scope	Public success card only	**Dual State**: Green (Authentic) vs. Red (Tampered/Hash Mismatch + 1-click FSSAI report)
Hardware / Seal	Basic QR sticker	Dual verification: Dynamic QR fallback + Web NFC (NTAG 424 DNA simulation)

Database Schema Blueprint (Supabase PostgreSQL)

Execute the following migrations in your Supabase SQL Editor. This schema supports role-based access control (RBAC), hive scale telemetry, mass-balance audits, and tamper flags.

```
-- 1. Enums
CREATE TYPE user_role AS ENUM ('beekeeper', 'lab_auditor', 'regulator', 'consumer');
CREATE TYPE hive_health_status AS ENUM ('healthy', 'mite_risk', 'inspection_needed');
CREATE TYPE batch_audit_status AS ENUM ('authentic', 'tampered', 'under_review');

-- 2. User Profiles
CREATE TABLE profiles (
    id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
    full_name TEXT NOT NULL,
    role user_role DEFAULT 'beekeeper',
    organization TEXT, -- e.g., 'ICAR-KVK Kanchipuram' or individual apiary
    phone TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- 3. Apiaries & Hives
CREATE TABLE apiaries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    beekeeper_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    location_name TEXT NOT NULL, -- e.g., 'Kanchipuram, TN'
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    is_reference_station BOOLEAN DEFAULT FALSE -- Flag for ICAR/KVK reference sites
);

CREATE TABLE hives (
    id TEXT PRIMARY KEY, -- e.g., 'HIVE-12'
    apiary_id UUID REFERENCES apiaries(id) ON DELETE CASCADE,
    current_weight_kg NUMERIC(6, 2) NOT NULL DEFAULT 0.0,
    ambient_temp_c NUMERIC(4, 1),
    ambient_humidity NUMERIC(4, 1),
    health_status hive_health_status DEFAULT 'healthy',
    last_weight_delta_kg NUMERIC(5, 2) DEFAULT 0.0,
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. Feeding Records (To prevent sugar syrup dilution fraud)
CREATE TABLE feeding_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    hive_id TEXT REFERENCES hives(id) ON DELETE CASCADE,
    feed_type TEXT NOT NULL, -- 'sugar_syrup', 'pollen_substitute'
    feed_mass_kg NUMERIC(5, 2) NOT NULL,
    fed_at TIMESTAMPTZ DEFAULT NOW()
);

-- 5. Honey Batches & Evidence Records
CREATE TABLE honey_batches (
    batch_id TEXT PRIMARY KEY, -- e.g., 'HC-0421'
    hive_id TEXT REFERENCES hives(id),
    harvest_weight_kg NUMERIC(6, 2) NOT NULL,
    calculated_mass_balance BOOLEAN DEFAULT TRUE, -- False if harvest mass exceeds hive yield delta
    lab_report_url TEXT,
    moisture_percentage NUMERIC(4, 2), -- Target < 18.0%
    is_fssai_compliant BOOLEAN DEFAULT TRUE,

    -- Evidence & Chain Anchoring
    merkle_root TEXT NOT NULL,
    blockchain_tx_hash TEXT,
    audit_status batch_audit_status DEFAULT 'authentic',
    tamper_reason TEXT, -- Populated if hash mismatch or seal broken

    harvested_at DATE NOT NULL DEFAULT CURRENT_DATE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 6. Retained Samples & Lab Splits
CREATE TABLE retained_samples (
    sample_id TEXT PRIMARY KEY, -- e.g., 'VIAL-HC-0421-RET'
    batch_id TEXT REFERENCES honey_batches(batch_id) ON DELETE CASCADE,
    nfc_tag_uid TEXT UNIQUE,
    cold_storage_location TEXT DEFAULT '4C Vault, KVK Station',
    custody_status TEXT DEFAULT 'sealed_intact'
);
```

Backend Evidence Engine (Bun + Hono)

Set up a Bun backend (`bun add hono @supabase/supabase-js ethers`) that implements HoneyChain's core differentiator: validating mass-balance before generating a batch anchor.

```
// src/index.ts
import { Hono } from 'hono';
import { cors } from 'hono/cors';
import { createClient } from '@supabase/supabase-js';
import { ethers } from 'ethers';

const app = new Hono();
app.use('*', cors());

const supabase = createClient(
```

```
  process.env.SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

// Mass-Balance Calculation & Batch Anchor Endpoint
app.post('/api/batches/create', async (c) => {
  const body = await c.req.json();
  const { hiveId, harvestWeightKg, moisture, labReportUrl } = body;

  // 1. Fetch Hive Telemetry & Feeding History
  const { data: hive } = await supabase
    .from('hives')
    .select('current_weight_kg, last_weight_delta_kg')
    .eq('id', hiveId)
    .single();

  const { data: feeds } = await supabase
    .from('feeding_logs')
    .select('feed_mass_kg')
    .eq('hive_id', hiveId);

  const totalFeedKg = (feeds || []).reduce((acc, f) => acc + Number(f.feed_mass_kg), 0);

  // 2. Evidence Rule: Harvest cannot exceed physical weight loss minus syrup conversion
  const allowableYield = Math.max(0, Number(hive?.last_weight_delta_kg || 0) + 1.0); // 1kg margin
  const massBalancePassed = harvestWeightKg <= allowableYield;

  // 3. Compute Merkle Leaf Hash
  const batchId = `HC-${Math.floor(1000 + Math.random() * 9000)}`;
  const payloadToHash = `${batchId}:${hiveId}:${harvestWeightKg}:${moisture}:${massBalancePassed}`;
  const leafHash = ethers.keccak256(ethers.toUtf8Bytes(payloadToHash));

  // 4. Insert Batch into Supabase
  const { data: batch, error } = await supabase
    .from('honey_batches')
    .insert({
      batch_id: batchId,
      hive_id: hiveId,
      harvest_weight_kg: harvestWeightKg,
      calculated_mass_balance: massBalancePassed,
      moisture_percentage: moisture,
      lab_report_url: labReportUrl,
      merkle_root: leafHash,
      audit_status: massBalancePassed ? 'authentic' : 'tampered',
      tamper_reason: massBalancePassed ? null : 'Mass-balance violation: Harvest exceeds biological hive delta
    })
    .select()
    .single();

  if (error) return c.json({ error: error.message }, 400);
  return c.json({ success: true, batch });
});

// Consumer Verification Endpoint with Tamper Simulation
app.get('/api/verify/:batchId', async (c) => {
  const batchId = c.req.param('batchId');

  const { data: batch, error } = await supabase
    .from('honey_batches')
    .select('*, hives(apiary_id, apiaries(location_name))')
    .eq('batch_id', batchId)
    .single();

  if (error || !batch) {
    return c.json({ status: 'not_found' }, 404);
  }

  return c.json({
    batchId: batch.batch_id,
    origin: batch.hives?.apiaries?.location_name || 'Kanchipuram, TN',
    harvestedDate: batch.harvested_at,
    moisture: `${batch.moisture_percentage}%`,
    status: batch.audit_status, // 'authentic' or 'tampered'
    tamperDetails: batch.tamper_reason,
    merkleRoot: batch.merkle_root,
```

```
    checks: {
      hiveVerified: true,
      labReportVerified: batch.is_fssai_compliant,
      jarSealedVerified: batch.audit_status === 'authentic',
      storeShelfVerified: batch.audit_status === 'authentic'
    }
  });
});

export default {
  port: 3001,
  fetch: app.fetch,
};
```

Frontend UI Layouts & Core Screens (React)

Recreate the screens outlined in Slide 4 of the project specifications, applying the visual polish and design system of the demo.

```
  Hive Monitor (/hives)           Verification Success (/verify)         Tamper Alert (/verify)
┌──────────────────────┐        ┌──────────────────────┐        ┌──────────────────────┐
│  Hive #12      +1.8 kg │      │  [ OK ] Authentic Honey │      │  [ X ] Tampered / Fake │
│  Hive #9       +1.1 kg │      │  Batch: HC-0421       │      │  Batch: HC-0999       │
│  Hive #7     Mite Risk │      │  Origin: Kanchipuram  │      │  Fail: Step 3 (Lab)   │
│  -------------------- │      │  --------------------  │      │  --------------------  │
│  [!] AI Alert: Inspect │      │  ✓ Hive Verified      │      │  Do not buy. Report it │
│      Hive #7 for mites │      │  ✓ Lab Report Verified │      │  [ Report to FSSAI ]  │
└──────────────────────┘        └──────────────────────┘        └──────────────────────┘
```

1. Hive Telemetry & Alert Screen (`/hives`)

Displays live load-cell weight, mass additions, and AI vision health alerts.

```
import React from 'react';
import { AlertTriangle, Scale, Activity } from 'lucide-react';

export const HiveMonitor = () => {
  const hives = [
    { id: 'Hive #12', delta: '+1.8 kg', status: 'optimal', color: 'bg-amber-100 text-amber-900' },
    { id: 'Hive #9',  delta: '+1.1 kg', status: 'optimal', color: 'bg-amber-100 text-amber-900' },
    { id: 'Hive #7',  delta: 'Mite risk', status: 'warning', color: 'bg-amber-50 text-amber-800' },
    { id: 'Hive #3',  delta: '+0.9 kg', status: 'optimal', color: 'bg-amber-100 text-amber-900' },
  ];

  return (
    <div className="max-w-md mx-auto p-4 bg-slate-50 min-h-screen">
      <div className="bg-sky-900 text-white p-4 rounded-xl text-center font-bold text-lg mb-4 shadow">
        My Hives Telemetry
      </div>

      <div className="space-y-3">
        {hives.map((hive) => (
          <div key={hive.id} className={`${hive.color} p-4 rounded-xl flex justify-between items-center shadow
            <div className="flex items-center gap-2 font-semibold">
              <span className="w-2.5 h-2.5 rounded-full bg-emerald-500"></span>
              {hive.id}
            </div>
            <span className="font-mono font-medium">{hive.delta}</span>
          </div>
        ))}
      </div>

      {/* AI Alert Card */}
      <div className="mt-6 border-2 border-rose-300 bg-rose-50 rounded-xl p-4 flex items-center gap-3">
        <AlertTriangle className="text-rose-600 w-8 h-8 flex-shrink-0" />
        <div className="text-sm text-rose-950 font-medium">
          <strong>AI Alert:</strong> Mite risk in Hive #7. Inspect frames for Varroa destructor.
        </div>
      </div>
    </div>
  );
};
```

2. Dual-State Consumer Verification Screen (`/verify?batch=...`)

Demonstrates authentic certification alongside tamper detection (e.g., simulating `HC-0999` with a hash mismatch).

```
import React, { useEffect, useState } from 'react';
import { CheckCircle2, XCircle, ShieldAlert, FileText, ArrowRight } from 'lucide-react';

export const ConsumerVerify = ({ batchId = 'HC-0421' }: { batchId?: string }) => {
  const [data, setData] = useState<any>(null);

  useEffect(() => {
    fetch(`http://localhost:3001/api/verify/${batchId}`)
      .then((res) => res.json())
      .then((json) => setData(json));
  }, [batchId]);

  if (!data) return <div className="p-8 text-center text-gray-500">Verifying ledger proof...</div>;

  const isAuthentic = data.status === 'authentic';

  return (
    <div className="max-w-md mx-auto p-4 min-h-screen bg-slate-50">
      <div className="bg-sky-900 text-white p-4 rounded-xl text-center font-bold text-lg shadow">
        Scan Result
      </div>

      {isAuthentic ? (
        /* PASS: AUTHENTIC HONEY VIEW */
        <div className="mt-4 bg-white rounded-2xl p-6 shadow-sm border border-slate-100">
          <div className="bg-emerald-100 text-emerald-800 text-center py-3 rounded-xl font-bold flex items-cer
            <CheckCircle2 className="w-6 h-6 text-emerald-600" />
            OK: Authentic Honey
          </div>

          <div className="space-y-2 text-sm border-b pb-4 mb-4">
            <div className="flex justify-between"><span className="text-gray-500">Batch</span><span className=
            <div className="flex justify-between"><span className="text-gray-500">Origin</span><span classNam
            <div className="flex justify-between"><span className="text-gray-500">Harvested</span><span classN
            <div className="flex justify-between"><span className="text-gray-500">Moisture</span><span classNa
          </div>

          {/* 4-Point Auditing Trace */}
          <div className="space-y-3">
            {[
              { label: 'Hive - verified', status: data.checks.hiveVerified },
              { label: 'Lab report - verified', status: data.checks.labReportVerified },
              { label: 'Jar sealed - verified', status: data.checks.jarSealedVerified },
              { label: 'Store shelf - verified', status: data.checks.storeShelfVerified },
            ].map((check, i) => (
              <div key={i} className="flex items-center gap-3 text-sm font-medium text-gray-700">
                <span className="w-3.5 h-3.5 rounded-full bg-emerald-500 flex-shrink-0"></span>
                {check.label}
              </div>
            ))}
          </div>
        </div>
      ) : (
        /* FAIL: TAMPER DETECTED VIEW */
        <div className="mt-4 bg-white rounded-2xl p-6 shadow-sm border border-rose-100">
          <div className="bg-rose-100 text-rose-800 text-center py-3 rounded-xl font-bold flex items-center ju
            <XCircle className="w-6 h-6 text-rose-600" />
            Tampered or Fake
          </div>

          <div className="space-y-2 text-sm border-b pb-4 mb-4">
            <div className="flex justify-between"><span className="text-gray-500">Batch</span><span className=
            <div className="flex justify-between"><span className="text-gray-500">Status</span><span classNam
            <div className="flex justify-between"><span className="text-gray-500">Failed at</span><span classN
          </div>

          <div className="bg-rose-50 text-rose-900 text-xs p-3 rounded-lg mb-6 leading-relaxed">
            This jar cannot be cryptographically verified against the Polygon ledger. The sealed sample recorc
          </div>

          <button
            onClick={() => alert('Incident logged with FSSAI portal!')}
            className="w-full bg-rose-600 hover:bg-rose-700 text-white font-semibold py-3 rounded-xl transitic
```

```
          >
            <ShieldAlert className="w-5 h-5" />
            Report to FSSAI
          </button>
        </div>
      )}
    </div>
  );
};
```

Step-by-Step Implementation Roadmap

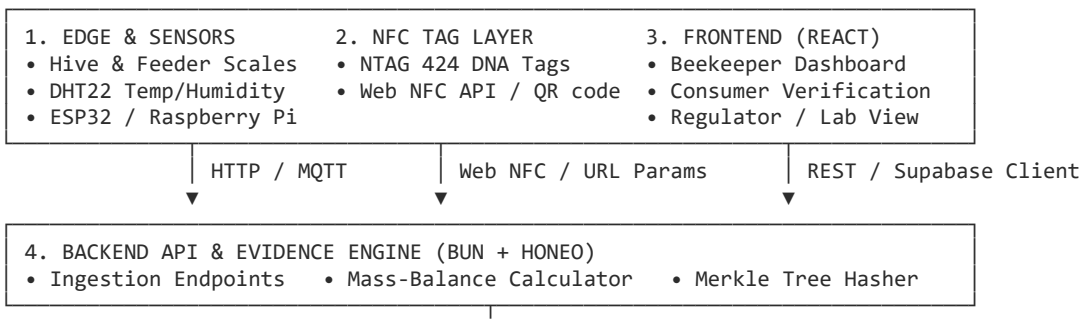
```
Week 1: Foundations & DB ⟶ Week 2: Evidence Engine ⟶ Week 3: Frontends ⟶ Week 4: Polish & Hackathon Demo
• Bun + Vite + React init    • Mass-balance calculation  • Hive telemetry view   • Preset quick-demo toggles
• Supabase migrations & RLS  • Merkle tree generator     • Verification cards    • Lab report PDF uploads
• Seed Kanchipuram apiary    • Amoy testnet contract     • Tamper simulation UI  • Presentation walkthrough
```

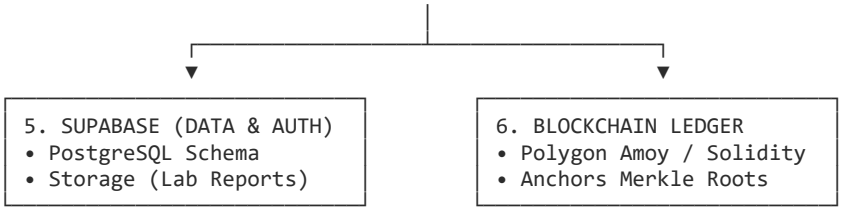
1. **Environment Setup & Database Seeding**
    - Initialize client and server using Bun (`bun create vite client --template react-ts`).
    - Apply the PostgreSQL schema in Supabase.
    - Seed `apiaries` with two locations: an ICAR-KVK Reference Station and an individual apiary (e.g., Kanchipuram, TN).
    - Seed baseline hive entries (`HIVE-12`, `HIVE-9`, `HIVE-7`).
2. **Core Logic: Mass-Balance & Merkle Batching**
    - Implement the endpoint `/api/batches/create` in Bun.
    - Ensure that attempting to log an inflated harvest weight triggers a `calculated_mass_balance = false` flag in Supabase, switching the batch status to `tampered`.
    - Deploy the minimal Solidity contract to Polygon Amoy Testnet (Chain ID 80002) to record `(batchId, merkleRoot, timestamp)`.
3. **Frontend Views (4 Prototype Screens)**
    - Build the **Hive Monitor** tab: Display weight deltas and trigger visual alert cards for `HIVE-7`.
    - Build the **Batch Creation & Label** view: Generate dynamic QR codes linking to `/verify?batch=<ID>`.
    - Build the **Public Verification Portal**: Render the 4-step verified status cards.
    - Build the **Tamper Simulation Mode**: Add a demo toggle button on the verification screen to switch between `HC-0421` (Authentic) and `HC-0999` (Tampered / Failed at Lab Step) to show judges how the system halts counterfeit goods in real time.
4. **Reference Station & Institutional Value Presentation**
    - Create an institutional read-only tab (`/reference-data`) showing real open datasets (pollen spectrum, moisture standards, isotopic signatures).
    - This ties directly into the project's primary institutional value: providing ICAR and KVIC with verified baseline datasets.

User prompt: so what are the different parts i need to build like , backend (api) , frontend for web access, code for nfc module (for tag writing and reading) , connection between the web app and nfc module , connection between web app and sensors etc

Response: To build the full **HoneyChain** evidence and traceability system using Bun, React, and Supabase, you need to implement five interconnected subsystems.

System Architecture Map





Part 1: IoT & Edge Sensing Subsystem (Hives & Feeders)

This layer captures physical yard metrics to verify biological honey accumulation and feed manipulation before extraction.

- **Hardware Components:**
  - **Hive Scale:** Load-cell bar (HX711 amplifier module) placed beneath the hive.
  - **Feeder Scale:** Load cell under artificial syrup feeders to monitor sugar dilution.
  - **Environmental Sensors:** DHT22 or SHT31 for interior temperature and humidity.
  - **Microcontroller:** ESP32 (Wi-Fi/Bluetooth) or a Raspberry Pi gateway running LoRa for remote apiaries.
- **Code to Build (Firmware):**
  - An Arduino/C++ or MicroPython sketch that reads the load-cell calibration factor every 15–30 minutes, packages readings into JSON (`{ hive_id, weight_kg, temp_c, humidity }`), and sends an authenticated HTTP `POST` to your Bun backend.

Part 2: NFC Module & Hardware Tagging Subsystem

HoneyChain uses cryptographic NFC tags (specifically **NTAG 424 DNA TagTamper**) to prevent post-harvest jar substitution and unsealed distribution.

- **Hardware / Tags:**
  - **NTAG 424 DNA Stickers:** Features a physical tamper loop that changes its cryptographic response when the jar lid seal is opened.
  - **Encoding Hardware:** A desktop USB NFC reader/writer (such as an ACR122U) or an NFC-enabled Android phone.
- **Code to Build (Tag Writer / Provisioning):**
  - A desktop CLI utility (in Python using `nfcpy` or Node/Bun using `nfc-pcsc`) to configure blank tags.
  - Writes an NDEF record encoding an authenticated redirect URL: `[https://honeychain.org/verify?tag_uid=04A1B2C3D4E5F6&cmac=E2849F](https://honeychain.org/verify?tag_uid=04A1B2C3D4E5F6&cmac=E2849F)...`
  - Programs the AES key into the tag so it generates dynamic Sun (Secure Unique NFC) tokens on every tap.

Part 3: Connections & Bridge Protocols

How Sensors Connect to the Web App & Backend

```
[ESP32 / Pi Gateway] ──(HTTP POST or MQTT)──► [Bun Backend] ──► [Supabase DB] ──► [React Web App]
```

1. **Sensors → Backend:** The microcontroller transmits sensor telemetry directly to a Bun API webhook: `POST /api/telemetry/record`.
2. **Backend → Database:** Bun writes readings directly into the Supabase `hives` and `feeding_logs` tables.
3. **Database → Frontend:** The React frontend connects via the `@supabase/supabase-js` client using **Supabase Realtime Subscriptions** (`supabase.channel('hives')`) so live weight fluctuations and AI health status render in real time without refreshing the page.

How NFC Connects to the Web App

```
[Physical Jar Tap] ──(Web NFC API or Browser Launch)──► [React /verify Page] ──► [Bun CMAC Decryption]
```

- **Mobile / Consumer Tap (Zero App Installation):** Tapping an NTAG 424 tag on an Android or iOS device opens the browser directly to `/verify?tag_uid=...`.
- **Web App In-Browser Reading (Web NFC API):** For browser-based scanning on supported Android Chrome devices, write a React hook:

```
const scanNfcTag = async () => {
  if ('NDEFReader' in window) {
```

```
      const ndef = new (window as any).NDEFReader();
      await ndef.scan();
      ndef.onreading = (event: any) => {
        const serialNumber = event.serialNumber;
        window.location.href = `/verify?tag_uid=${serialNumber}`;
      };
    } else {
      // Fallback to QR camera scanner
    }
  };
```

- **Desktop Station Reading (Reference Stations / Labs):** If an ICAR/KVK lab uses a USB NFC writer, write a local daemon script running via Bun/Python that communicates with the local browser via WebSockets (`ws://localhost:8080`) to pass tag IDs to the input form automatically.

Part 4: Backend API & Evidence Engine (Bun + Supabase)

The backend coordinates data validation and ensures no fraudulent honey enters the supply chain.

- **Bun API Gateway (using Hono or Elysia):**
  - `POST /api/telemetry/record`: Ingests hive weight and feed mass.
  - `POST /api/batches/create`: Computes mass-balance, generates the Merkle leaf, and flags anomalies.
  - `GET /api/verify/:batchId`: Validates jar parameters and cross-checks ledger proofs.
  - `POST /api/nfc/validate-cmac`: Decrypts dynamic NTAG 424 DNA tokens to confirm the tag is original and unopened.
- **The Evidence Engine (Core Differentiator):**
  - **Mass-Balance Audit:** Implements the formula:

    $$\text{Harvest Mass (kg)} \leq \Delta\text{Hive Weight (kg)} - (\text{Feed Input} \times \text{Conversion Ratio})$$

    If harvest mass exceeds the allowable delta, it automatically flags the batch as `tampered` due to syrup adulteration.
  - **Merkle Batcher:** Combines `[batchId, apiaryId, harvestWeight, labHash]` into a Merkle tree root.
- **Ledger Anchor Worker:**
  - Uses `ethers.js` or `viem` to broadcast the Merkle root to a Solidity smart contract on the Polygon Amoy testnet or Hyperledger Besu.

Part 5: Frontend Web Application (React + Vite + Tailwind)

Build four primary role-based views in React:

Screen / Route	Core Features	Inspired By
Hive Telemetry (`/hives`)	Live load-cell weight cards (+1.8 kg, +1.1 kg), AI brood mite risk badges, feeding logs.	HoneyChain Slide 4 & 5
Batch Creator (`/batch/new`)	Batch ID input, scale weight capture, split retained sample logger, label generator.	HoneyChain Slide 3 & 4
Consumer Verification (`/verify`)	Dual-state display: **Green Authentic** (4 verification stages) vs. **Red Tampered / Fake** with 1-click FSSAI report.	HoneyChain Slide 4 & Bee Bridge
Lab & Reference Station (`/reference`)	ICAR/KVK reference datasets, pollen spectra, NMR/IRMS test report PDF uploads.	HoneyChain Slide 5

Part 6: AI Brood Health Microservice (MobileNetV2)

Identifies Varroa destructor mites and colony decline:

- **Model:** Lightweight TensorFlow MobileNetV2 trained on apiary brood frames.
- **Execution Options:**
  1. **Client-Side:** Load the converted model via `@tensorflow/tfjs` in React to classify uploaded photos directly in the browser without server overhead.
  2. **Backend:** Host an inference endpoint running Python FastAPI or a Bun worker that returns `{ classification: "healthy" | "mite_risk", confidence: 0.94 }`.

Would you like to start with the **firmware code for the ESP32 sensors**, the **Bun backend's mass-balance validation logic**, or the **React consumer verification page**?
